

ĐẠI HỌC BÁCH KHOA HÀ NỘI  
Trường Công nghệ Thông tin và Truyền thông

## BÁO CÁO BÀI TẬP LỚN

Học phần: Nhập môn Trí tuệ nhân tạo

Mã học phần: IT3160 Mã lớp học: 167855

Đề tài:

**Phân loại cảm xúc đa mức trên Amazon Reviews  
bằng mô hình BiLSTM kết hợp Attention**

|                           |                                |
|---------------------------|--------------------------------|
| Giảng viên hướng dẫn:     | PGS. TS. Ngô Văn Linh          |
| Nhóm sinh viên thực hiện: | Trần Lê Ngọc Tâm 202416346     |
|                           | Nguyễn Phong 202400066         |
|                           | Nguyễn Ngọc Tuấn Anh 202400029 |

Hà Nội, Tháng 06 Năm 2026

# Mục lục

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>ABSTRACT</b>                                           | <b>5</b>  |
| <b>1 Giới thiệu và phát biểu bài toán</b>                 | <b>6</b>  |
| 1.1 Bối cảnh                                              | 6         |
| 1.2 Mục tiêu                                              | 6         |
| 1.3 Phạm vi và nguyên tắc đọc kết quả                     | 7         |
| 1.4 Đóng góp chính                                        | 7         |
| <b>2 Dữ liệu và pipeline thực nghiệm</b>                  | <b>8</b>  |
| 2.1 Nguồn dữ liệu                                         | 8         |
| 2.2 Phân bố lớp                                           | 8         |
| 2.3 Làm sạch và biểu diễn văn bản                         | 9         |
| 2.4 Chia dữ liệu và chống leakage                         | 9         |
| 2.5 Cấu hình huấn luyện chung                             | 10        |
| 2.6 Các độ đo đánh giá                                    | 10        |
| <b>3 Mô hình chính: BiLSTM kết hợp Additive Attention</b> | <b>12</b> |
| 3.1 Lý do lựa chọn kiến trúc                              | 12        |
| 3.2 Sơ đồ kiến trúc tổng thể                              | 12        |
| 3.3 Kích thước tensor và số tham số                       | 14        |
| 3.4 Cơ chế LSTM                                           | 14        |
| 3.5 Biểu diễn hai chiều                                   | 15        |
| 3.6 Additive attention có mask                            | 15        |
| 3.7 Classifier và regularization                          | 16        |
| 3.8 Diễn biến huấn luyện                                  | 17        |
| 3.9 Confusion matrix của mô hình chính                    | 18        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 3.10     | Điểm mạnh và giới hạn nội tại . . . . .                            | 19        |
| <b>4</b> | <b>Hai mô hình đối chứng</b>                                       | <b>20</b> |
| 4.1      | SimpleRNN baseline . . . . .                                       | 20        |
| 4.2      | Transformer Encoder baseline . . . . .                             | 21        |
| 4.3      | Điều kiện so sánh . . . . .                                        | 21        |
| 4.4      | Tóm tắt khác biệt kiến trúc . . . . .                              | 22        |
| <b>5</b> | <b>Kết quả và so sánh chi tiết ba mô hình</b>                      | <b>23</b> |
| 5.1      | Kết quả tổng hợp . . . . .                                         | 23        |
| 5.2      | So sánh F1 theo từng mức rating . . . . .                          | 25        |
| 5.3      | So sánh confusion matrix . . . . .                                 | 26        |
| 5.4      | Đường validation của ba mô hình . . . . .                          | 27        |
| 5.5      | Chi phí tài nguyên . . . . .                                       | 28        |
| 5.6      | So sánh trực tiếp BiLSTM + Attention với SimpleRNN . . . . .       | 28        |
| 5.7      | So sánh trực tiếp BiLSTM + Attention với Transformer . . . . .     | 29        |
| 5.8      | Mô hình được lựa chọn . . . . .                                    | 29        |
| <b>6</b> | <b>Vấn đề tồn tại, hướng cải thiện và kết luận</b>                 | <b>30</b> |
| 6.1      | Vấn đề nghiêm trọng nhất: mất cân bằng và sụp đổ lớp . . . . .     | 30        |
| 6.2      | Bài toán có thứ bậc nhưng loss đang xem nhãn là danh mục . . . . . | 30        |
| 6.3      | Hạn chế của quy trình đánh giá . . . . .                           | 31        |
| 6.4      | Hạn chế tiền xử lý . . . . .                                       | 31        |
| 6.5      | Hạn chế riêng của BiLSTM + Attention . . . . .                     | 32        |
| 6.6      | Hạn chế riêng của hai baseline . . . . .                           | 32        |
| 6.7      | Lộ trình cải thiện đề xuất . . . . .                               | 33        |
| 6.8      | Kết luận . . . . .                                                 | 33        |
|          | <b>TÀI LIỆU THAM KHẢO</b>                                          | <b>34</b> |

# Danh sách hình vẽ

|     |                                                                             |    |
|-----|-----------------------------------------------------------------------------|----|
| 3.1 | Kiến trúc chi tiết của mô hình BiLSTM + Attention được sử dụng . . . . .    | 13 |
| 3.2 | Loss và Accuracy theo epoch của BiLSTM + Attention . . . . .                | 17 |
| 3.3 | Confusion matrix của BiLSTM + Attention trên 4.081 mẫu validation . . . . . | 18 |
| 5.1 | So sánh các metric chất lượng tổng hợp . . . . .                            | 24 |
| 5.2 | F1-score theo từng lớp rating . . . . .                                     | 25 |
| 5.3 | Confusion matrix của ba mô hình . . . . .                                   | 26 |
| 5.4 | Validation loss và validation accuracy của ba mô hình . . . . .             | 27 |
| 5.5 | Training history chi tiết của hai mô hình đối chứng . . . . .               | 27 |
| 5.6 | So sánh số tham số, thời gian train và thời gian inference . . . . .        | 28 |

# Danh sách bảng

|     |                                                                     |    |
|-----|---------------------------------------------------------------------|----|
| 2.1 | Phân bố nhãn trên tập validation . . . . .                          | 8  |
| 2.2 | Cấu hình thực tế lấy từ các tệp <code>metrics.json</code> . . . . . | 10 |
| 3.1 | Chi tiết từng lớp của mô hình chính . . . . .                       | 14 |
| 4.1 | So sánh định tính ba kiến trúc . . . . .                            | 22 |
| 5.1 | Kết quả trên cùng tập validation 4.081 mẫu . . . . .                | 23 |
| 5.2 | F1-score từng lớp . . . . .                                         | 25 |
| 5.3 | Phân bố nhãn dự đoán của BiLSTM + Attention . . . . .               | 26 |
| 5.4 | Số tham số và thời gian ghi nhận trong artifact . . . . .           | 28 |
| 6.1 | Thứ tự ưu tiên cải thiện project . . . . .                          | 33 |

# ABSTRACT

Báo cáo trình bày bài toán phân loại đánh giá sản phẩm Amazon thành năm mức từ 1 đến 5 sao. Trọng tâm của nghiên cứu là kiến trúc **Bidirectional Long Short-Term Memory kết hợp additive attention có mask padding**. Hai mô hình SimpleRNN và Transformer Encoder nhỏ được cài đặt trên cùng pipeline, cùng cách chia dữ liệu và cùng không gian từ vựng để đóng vai trò đối chứng, không phải là đối tượng chính của đề tài.

Pipeline sử dụng 20.405 review hợp lệ, chia phân tầng 80% cho huấn luyện và 20% cho validation. Văn bản được chuẩn hóa, token hóa bằng từ vựng tối đa 50.000 từ, cắt hoặc đệm về 200 token và biểu diễn bằng GloVe 100 chiều có thể tiếp tục cập nhật. Mô hình chính gồm Embedding, Dropout, BiLSTM 128 đơn vị trên mỗi chiều, additive attention trên vector trạng thái 256 chiều, Dense 64 nút và Softmax 5 lớp. Kết quả trên 4.081 mẫu validation cho thấy BiLSTM + Attention đạt Accuracy 81,38%, Macro F1 0,3727, Weighted F1 0,7549 và rating MAE 0,3328; tốt hơn cả SimpleRNN và Transformer trong lần chạy được lưu.

Tuy vậy, kết quả cũng cho thấy một hạn chế nghiêm trọng: dữ liệu lệch mạnh về lớp 1 sao và class weights đã bị tắt trong cả ba lần chạy. SimpleRNN và Transformer không dự đoán đúng bất kỳ mẫu 2, 3 hoặc 4 sao nào. BiLSTM + Attention có tạo dự đoán cho các lớp giữa nhưng recall vẫn rất thấp, lần lượt chỉ 0,83%, 3,64% và 3,77%. Do đó, Accuracy cao không đồng nghĩa với hệ thống đã giải quyết tốt bài toán năm lớp. Báo cáo phân tích thẳng thắn hiện tượng mô hình co cụm về hai cực 1 và 5 sao, các giới hạn của quy trình đánh giá hiện tại, và đề xuất hướng cải thiện dựa trên cân bằng lớp, loss có tính thứ bậc, tập test độc lập và phân tích attention.

**Keywords:** sentiment analysis, Amazon Reviews, BiLSTM, additive attention, class imbalance, ordinal classification, Transformer.

# Chương 1

## Giới thiệu và phát biểu bài toán

### 1.1 Bối cảnh

Đánh giá sản phẩm là nguồn dữ liệu quan trọng đối với cả người mua, nhà bán hàng và nền tảng thương mại điện tử. Khác với bài toán sentiment nhị phân chỉ phân biệt tích cực và tiêu cực, thang 1–5 sao chứa nhiều mức độ: 1 sao thường thể hiện bất mãn rõ rệt, 5 sao thể hiện hài lòng mạnh, trong khi 2–4 sao chứa những nhận xét pha trộn, trung tính hoặc có điều kiện. Việc nhận diện đúng các lớp giữa khó hơn đáng kể vì ranh giới ngữ nghĩa mờ và nhiều review đồng thời chứa cả ưu điểm lẫn nhược điểm.

Ví dụ, câu “The screen is excellent, but the battery is disappointing” chứa hai tín hiệu trái chiều. Một mô hình chỉ dựa vào một vài từ tích cực hoặc tiêu cực rất dễ đẩy review về 1 hoặc 5 sao. Mô hình cần vừa đọc được ngữ cảnh theo hai chiều, vừa xác định thành phần nào trong chuỗi có ảnh hưởng lớn hơn tới đánh giá cuối cùng. Đây là động cơ lựa chọn BiLSTM kết hợp Attention.

### 1.2 Mục tiêu

Các mục tiêu cụ thể của project gồm:

1. Xây dựng pipeline thống nhất để tiền xử lý Amazon Reviews và phân loại mỗi review vào một trong năm mức rating.
2. Cài đặt chi tiết mô hình chính BiLSTM + additive attention, trong đó padding được mask trước phép softmax của attention.
3. Dùng SimpleRNN và một Transformer Encoder nhỏ làm hai đường cơ sở để đánh giá lợi ích của ngữ cảnh hai chiều và attention.

4. So sánh không chỉ bằng Accuracy mà còn bằng Macro Precision, Macro Recall, Macro F1, Weighted F1, F1 từng lớp, confusion matrix, thời gian huấn luyện và thời gian suy luận.
5. Nhận diện các thất bại của mô hình thay vì chỉ báo cáo con số tổng hợp, đặc biệt là khả năng phân loại các lớp 2, 3 và 4 sao.

### 1.3 Phạm vi và nguyên tắc đọc kết quả

Ba mô hình được huấn luyện từ cùng một split và cùng biểu diễn GloVe. Transformer trong project là một encoder nhỏ huấn luyện cho bài toán hiện tại, **không phải** BERT hay một language model pretrained quy mô lớn. Vì vậy, so sánh trong báo cáo nhằm trả lời câu hỏi kiến trúc nào hoạt động tốt hơn trong đúng thiết lập này, không nhằm kết luận phổ quát rằng BiLSTM luôn tốt hơn Transformer.

Ngoài ra, tập được gọi là validation vừa tham gia điều khiển EarlyStopping vừa được dùng để báo cáo kết quả cuối cùng. Chưa có tập test độc lập. Các con số vì thế nên được xem là kết quả thực nghiệm nội bộ của project, chưa phải ước lượng cuối cùng về khả năng tổng quát hóa trên dữ liệu hoàn toàn chưa quan sát.

### 1.4 Đóng góp chính

Đóng góp cốt lõi của báo cáo là việc xây dựng và phân tích mô hình BiLSTM + Attention với các đặc điểm:

- Sử dụng GloVe 100 chiều được tinh chỉnh riêng cho miền Amazon.
- Triển khai BiLSTM với không gian trạng thái 256 chiều.
- Tích hợp Additive Attention có khả năng học và hỗ trợ mask padding.
- Áp dụng Dropout (0,5) và L2 regularization (0,01) để chống overfitting.
- Đánh giá toàn diện qua nhiều độ đo (Accuracy, F1) và tài nguyên.

Bên cạnh đó, báo cáo thẳng thắn phân tích nguyên nhân mô hình thất bại ở các lớp thiểu số (2-4 sao), qua đó khẳng định Accuracy cao không đồng nghĩa với việc bài toán đã được giải quyết trọn vẹn.



## Chương 2

# Dữ liệu và pipeline thực nghiệm

### 2.1 Nguồn dữ liệu

Project sử dụng Amazon Reviews Dataset trên Kaggle, handle `dongrelaxman/amazon-reviews-dataset`. Hai cột được đọc là `Review Text` và `Rating`. Sau khi bỏ giá trị thiếu, rating không hợp lệ, review rỗng và bản ghi trùng theo cặp văn bản–rating, pipeline thu được 20.405 mẫu hợp lệ.

Nhãn rating ban đầu thuộc tập  $\{1, 2, 3, 4, 5\}$  và được chuyển về  $\{0, 1, 2, 3, 4\}$  khi huấn luyện để phù hợp với `sparse_categorical_crossentropy`. Khi báo cáo, nhãn được đổi lại thành 1–5 sao.

### 2.2 Phân bố lớp

Tập validation có 4.081 mẫu và phản ánh rõ mức độ mất cân bằng của dữ liệu. Bảng 2.1 được lấy trực tiếp từ support trong classification report của ba mô hình.

Bảng 2.1: Phân bố nhãn trên tập validation

| Lớp         | Số mẫu       | Tỷ lệ       |
|-------------|--------------|-------------|
| 1 sao       | 2.591        | 63,49%      |
| 2 sao       | 241          | 5,91%       |
| 3 sao       | 165          | 4,04%       |
| 4 sao       | 239          | 5,86%       |
| 5 sao       | 845          | 20,71%      |
| <b>Tổng</b> | <b>4.081</b> | <b>100%</b> |

Nếu luôn dự đoán 1 sao, một baseline rất đơn giản đã đạt 63,49% Accuracy. Điều này cho thấy Accuracy phải được đọc cùng Macro F1 và recall từng lớp. Ba lớp 2–4 sao cộng lại chiếm 15,80%, nhưng lại là phần khó và có ý

nghĩa nhất để kiểm tra mô hình có thực sự học bài toán năm lớp hay chỉ tách hai cực.

## 2.3 Làm sạch và biểu diễn văn bản

Hàm `clean_text` thực hiện:

1. chuyển văn bản về chữ thường;
2. loại bỏ thẻ HTML;
3. giữ các ký tự a-z, chữ số, khoảng trắng và dấu nháy đơn;
4. gộp nhiều khoảng trắng liên tiếp;
5. loại review rỗng sau làm sạch.

Tokenizer Keras có giới hạn tối đa 50.000 từ và token `<OOV>`. Tokenizer chỉ được fit trên tập train, vì vậy vocabulary của validation không làm rò rỉ vào quá trình học. Tokenizer lưu trong artifact chứa 26.714 mục; cộng token padding tạo ra kích thước embedding thực tế  $V = 26.715$ . Mỗi chuỗi được post-padding hoặc post-truncation về  $T = 200$  token.

Ma trận embedding được khởi tạo từ GloVe 6B 100d [5]. Các từ khớp GloVe nhận vector pretrained; các từ không khớp bắt đầu bằng vector 0. Lớp embedding đặt `trainable=True`, nên toàn bộ vector tiếp tục được cập nhật trong quá trình huấn luyện. Token 0 được dùng cho padding và `mask_zero=True`.

## 2.4 Chia dữ liệu và chống leakage

Dữ liệu được chia train/validation theo tỷ lệ 80/20, stratify theo nhãn và dùng seed 42. Với 20.405 mẫu, kích thước tương ứng là 16.324 mẫu train và 4.081 mẫu validation. Thứ tự đúng của pipeline là:

làm sạch → chia train/validation → fit tokenizer trên train → biến đổi cả hai tập.

Cách làm này tốt hơn việc fit tokenizer trước khi chia dữ liệu, bởi trường hợp sau sẽ để vocabulary của validation ảnh hưởng gián tiếp tới biểu diễn đầu vào.

## 2.5 Cấu hình huấn luyện chung

Bảng 2.2: Cấu hình thực tế lấy từ các tệp `metrics.json`

| Thành phần                        | Giá trị                                                                   |
|-----------------------------------|---------------------------------------------------------------------------|
| <b>Số lớp</b>                     | 5                                                                         |
| <b>Độ dài chuỗi</b>               | 200 token                                                                 |
| <b>Giới hạn từ vựng</b>           | 50.000; kích thước thực tế 26.715                                         |
| <b>Embedding</b>                  | GloVe 6B, 100 chiều, có fine-tune                                         |
| <b>Batch size</b>                 | 128                                                                       |
| <b>Epoch tối đa</b>               | 10                                                                        |
| <b>Optimizer</b>                  | Adam, learning rate ban đầu 0,001                                         |
| <b>Loss</b>                       | Sparse categorical cross-entropy + L2 từ các lớp Dense                    |
| <b>Dropout</b>                    | 0,5                                                                       |
| <b>L2</b>                         | 0,01                                                                      |
| <b>Validation split</b>           | 0,2; stratified; seed 42                                                  |
| <b>EarlyStopping</b>              | Theo dõi <code>val_loss</code> , patience 3, khôi phục trọng số tốt nhất  |
| <b>ReduceLROnPlateau</b>          | Theo dõi <code>val_loss</code> , factor 0,5, patience 2, min LR $10^{-6}$ |
| <b>Class weights của lần chạy</b> | <b>Bật</b> ( <code>use_class_weights=true</code> )                        |

## 2.6 Các độ đo đánh giá

Với  $K = 5$  lớp, Accuracy được tính bởi

$$\text{Accuracy} = \frac{\sum_{i=1}^N \mathbb{I}(y_i = \hat{y}_i)}{N}.$$

Precision, Recall và F1 cho lớp  $k$  lần lượt là

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad R_k = \frac{TP_k}{TP_k + FN_k}, \quad F1_k = \frac{2P_k R_k}{P_k + R_k}.$$

Macro F1 xem mọi lớp có trọng số như nhau:

$$\text{MacroF1} = \frac{1}{K} \sum_{k=1}^K F1_k.$$

Do đó, nếu mô hình bỏ qua lớp thiểu số, Macro F1 sẽ giảm mạnh ngay cả khi Accuracy còn cao. Weighted F1 dùng support của từng lớp làm trọng số, nên gần với chất lượng trên các lớp lớn hơn và dễ bị lớp 1 sao chi phối.

## Chương 3

# Mô hình chính: BiLSTM kết hợp Additive Attention

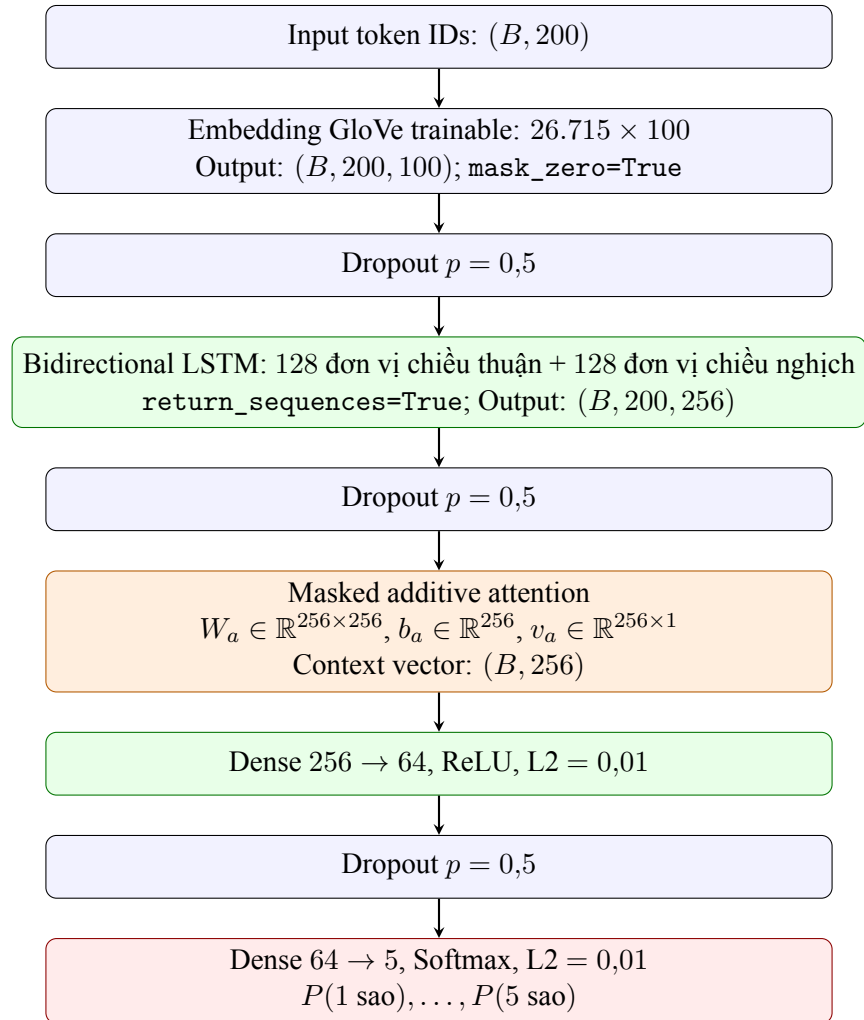
### 3.1 Lý do lựa chọn kiến trúc

SimpleRNN truyền một hidden state qua toàn chuỗi nên dễ suy giảm thông tin dài hạn. LSTM bổ sung cell state và các cổng điều khiển để giảm vấn đề vanishing gradient [1]. BiLSTM chạy hai LSTM theo hai hướng [2]: chiều thuận đọc từ trái sang phải, chiều nghịch đọc từ phải sang trái. Nhờ đó, biểu diễn tại một vị trí chứa cả ngữ cảnh trước và sau từ đang xét.

Tuy nhiên, nếu chỉ lấy trạng thái cuối, toàn bộ review vẫn phải bị nén vào một vector duy nhất. Additive attention [3] cho phép mô hình học mức quan trọng của từng timestep, rồi lấy tổng có trọng số. Đây là điểm phù hợp với sentiment analysis: các từ hoặc cụm như “not worth”, “works perfectly”, “but”, “disappointed” không đóng góp như nhau.

### 3.2 Sơ đồ kiến trúc tổng thể

Hình 3.1 biểu diễn đúng luồng tensor trong file BiLSTMAttention/BiLSTM\_Attention.py. Sơ đồ được vẽ trực tiếp bằng TikZ nên không cần thêm ảnh kiến trúc bên ngoài.



Hình 3.1: Kiến trúc chi tiết của mô hình BiLSTM + Attention được sử dụng

### 3.3 Kích thước tensor và số tham số

Bảng 3.1: Chi tiết từng lớp của mô hình chính

| Thứ tự      | Lớp                | Cấu hình                                    | Output shape    | Tham số          |
|-------------|--------------------|---------------------------------------------|-----------------|------------------|
| 1           | Input              | 200 token integer                           | $(B, 200)$      | 0                |
| 2           | Embedding          | $V = 26.715$ , $d = 100$ , GloVe, trainable | $(B, 200, 100)$ | 2.671.500        |
| 3           | Dropout            | $p = 0,5$                                   | $(B, 200, 100)$ | 0                |
| 4           | Bidirectional LSTM | 128 đơn vị mỗi chiều, return sequences      | $(B, 200, 256)$ | 234.496          |
| 5           | Dropout            | $p = 0,5$                                   | $(B, 200, 256)$ | 0                |
| 6           | Additive Attention | masked, một context vector                  | $(B, 256)$      | 66.048           |
| 7           | Dense              | 64, ReLU, L2 = 0,01                         | $(B, 64)$       | 16.448           |
| 8           | Dropout            | $p = 0,5$                                   | $(B, 64)$       | 0                |
| 9           | Dense              | 5, Softmax, L2 = 0,01                       | $(B, 5)$        | 325              |
| <b>Tổng</b> |                    |                                             |                 | <b>2.988.817</b> |

Riêng embedding chiếm 2.671.500 tham số, tương đương 89,38% tổng tham số mô hình. Vì thế, chênh lệch tổng số tham số giữa ba kiến trúc không lớn: phần lớn ngân sách tham số nằm ở bảng từ, không phải khối tuần tự.

### 3.4 Cơ chế LSTM

Tại timestep  $t$ , với vector đầu vào  $x_t$ , hidden state  $h_{t-1}$  và cell state  $c_{t-1}$ , một LSTM chuẩn tính:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad (3.1)$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad (3.2)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad (3.3)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad (3.4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (3.5)$$

$$h_t = o_t \odot \tanh(c_t). \quad (3.6)$$

Forget gate  $f_t$  đóng vai trò cốt lõi trong việc gạt bỏ các thông tin nhiễu từ quá khứ; input gate  $i_t$  điều khiển mức độ dung nạp nội dung mới; trong khi output gate  $o_t$  xác định phần cell state nào được phép phát ra. Điểm đột phá lớn nhất của kiến trúc LSTM nằm ở luồng truyền trạng thái  $c_t$ : việc sử dụng phép cộng ( $\oplus$ ) thay vì phép nhân ma trận liên tiếp giúp gradient có thể trượt qua các cell mà không bị triệt tiêu (vanishing gradient), khắc phục hoàn

toàn điểm yếu chỉ từ của SimpleRNN.

### 3.5 Biểu diễn hai chiều

Hai LSTM độc lập sinh:

$$\vec{h}_t = \text{LSTM}_{fwd}(x_t, \vec{h}_{t-1}), \quad \overleftarrow{h}_t = \text{LSTM}_{bwd}(x_t, \overleftarrow{h}_{t+1}).$$

Output tại vị trí  $t$  là phép nối

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \in \mathbb{R}^{256}.$$

Với tham số `return_sequences=True`, mô hình không chỉ nhả ra trạng thái cuối cùng mà trả về toàn bộ quỹ đạo ẩn  $H = [h_1, \dots, h_T] \in \mathbb{R}^{T \times 256}$ . Ma trận  $H$  này là nền tảng tối quan trọng để cơ chế Attention phía sau có thể quan sát, so sánh và trích xuất thông tin thay vì bị ép nén toàn bộ ngữ cảnh một cách thô bạo vào một vector cuối cùng.

### 3.6 Additive attention có mask

Attention layer của project học ba tham số:

$$W_a \in \mathbb{R}^{256 \times 256}, \quad b_a \in \mathbb{R}^{256}, \quad v_a \in \mathbb{R}^{256 \times 1}.$$

Với mỗi hidden state  $h_t$ , score chưa chuẩn hóa được tính:

$$u_t = \tanh(h_t W_a + b_a), \tag{3.7}$$

$$e_t = u_t v_a. \tag{3.8}$$

Đối với timestep padding, implementation thay  $e_t$  bằng  $-10^9$  trước softmax. Sau đó:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^T \exp(e_j)}, \tag{3.9}$$

$$c = \sum_{t=1}^T \alpha_t h_t. \tag{3.10}$$

Do  $\exp(-10^9) \approx 0$ , padding gần như nhận trọng số bằng 0. Context vector  $c \in \mathbb{R}^{256}$  là biểu diễn duy nhất

chuyển sang classifier. Tổng số tham số attention là

$$256^2 + 256 + 256 = 66.048.$$

---

**Algorithm 1** Masked additive attention trong mô hình chính

---

**Require:**  $H \in \mathbb{R}^{B \times T \times 256}$ , mask  $M \in \{0, 1\}^{B \times T}$

1:  $E \leftarrow \text{squeeze}(\tanh(HW_a + b_a)v_a)$

2:  $E \leftarrow \text{where}(M = 1, E, -10^9)$

3:  $A \leftarrow \text{softmax}(E, \text{axis} = T)$

4:  $C \leftarrow \sum_{t=1}^T A_t H_t$

5: **return**  $C$

---

### 3.7 Classifier và regularization

Context vector đi qua Dense 64 nút với ReLU:

$$z = \text{ReLU}(W_c c + b_c).$$

Lớp cuối sinh phân phối xác suất:

$$\hat{y} = \text{softmax}(W_o z + b_o).$$

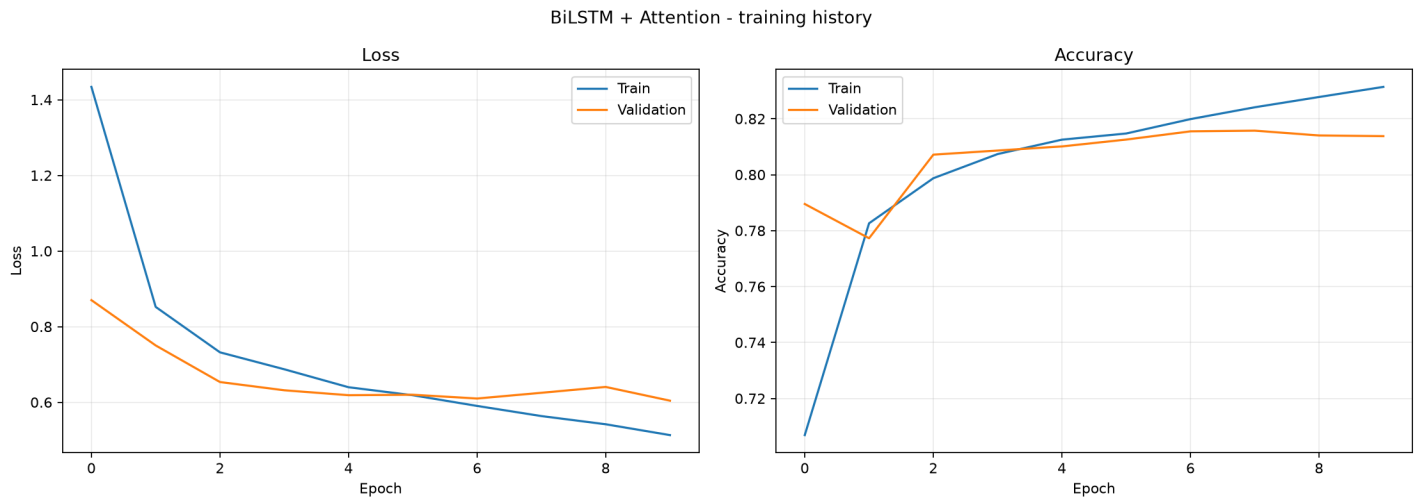
Hai kernel Dense có L2 regularization:

$$\mathcal{L} = \mathcal{L}_{CE} + \lambda (\|W_c\|_2^2 + \|W_o\|_2^2), \quad \lambda = 0,01.$$

Dropout 0,5 được đặt sau embedding, sau BiLSTM và sau Dense 64. Lưu ý đây là Dropout thông thường trong mã nguồn, không phải SpatialDropout1D. Việc ghi đúng chi tiết này quan trọng vì hai lớp có cách loại bỏ phần tử khác nhau.



### 3.8 Diễn biến huấn luyện

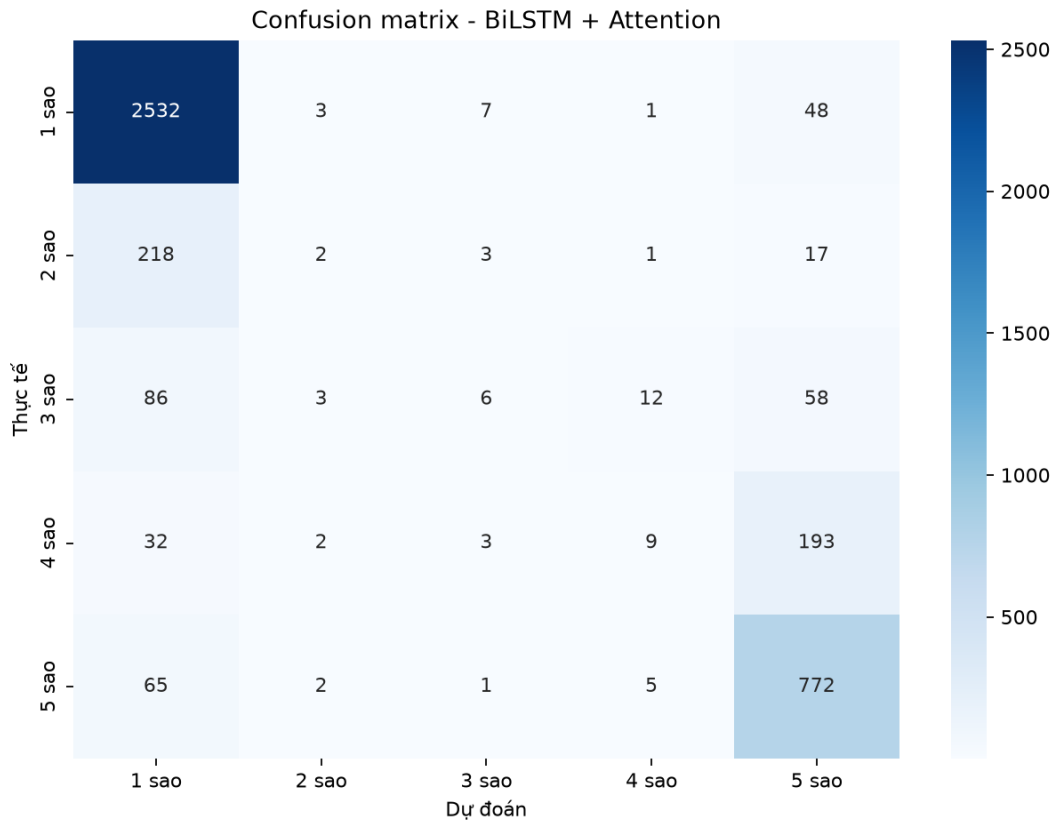


Hình 3.2: Loss và Accuracy theo epoch của BiLSTM + Attention

Mô hình chính huấn luyện đủ 10 epoch. Train accuracy tăng từ 70,69% lên 83,14%; validation accuracy tăng nhanh và ổn định quanh 81% từ epoch 3. Giá trị validation loss thấp nhất xuất hiện ở epoch cuối, khoảng 0,6042. Learning rate được giảm từ 0,001 xuống 0,0005 ở epoch cuối sau giai đoạn plateau.

Khoảng cách train-validation accuracy cuối cùng chỉ khoảng 1,76 điểm phần trăm, nên biểu đồ không cho thấy overfitting tổng thể quá mạnh. Tuy nhiên, kết luận này chỉ đúng với metric tổng hợp. Một mô hình có thể có đường validation đẹp nhưng vẫn thất bại ở lớp thiểu số; confusion matrix cho thấy đúng hiện tượng đó.

### 3.9 Confusion matrix của mô hình chính



Hình 3.3: Confusion matrix của BiLSTM + Attention trên 4.081 mẫu validation

Mô hình nhận diện rất tốt hai lớp cực trị:

- lớp 1 sao: recall 97,72%, F1 0,9167;
- lớp 5 sao: recall 91,36%, F1 0,7988.

Ngược lại, các lớp giữa rất yếu:

- 2 sao: chỉ đúng 2/241 mẫu; 218 mẫu bị đẩy về 1 sao;
- 3 sao: chỉ đúng 6/165 mẫu; 86 mẫu về 1 sao và 58 mẫu lên 5 sao;
- 4 sao: chỉ đúng 9/239 mẫu; 193 mẫu bị đẩy lên 5 sao.

Tổng cộng, tập validation có 645 mẫu thuộc 2–4 sao, nhưng mô hình chỉ tạo 60 dự đoán thuộc ba lớp này, tương đương 1,47% toàn bộ dự đoán. Trong khi đó, tỷ lệ thật của ba lớp giữa là 15,80%. Nói cách khác, BiLSTM + Attention đã học khá tốt trục sentiment tiêu cực–tích cực, nhưng chưa học tốt độ mịn năm mức.

### 3.10 Điểm mạnh và giới hạn nội tại

**Điểm mạnh** của kiến trúc là khả năng kết hợp ngữ cảnh hai chiều, xử lý chuỗi có thứ tự, mask padding đúng và tạo một cơ chế chọn timestep có thể mở rộng để giải thích dự đoán.

**Giới hạn** là toàn bộ chuỗi bị nén vào một context vector 256 chiều với một bộ trọng số attention duy nhất. Review có nhiều khía cạnh như giá, độ bền, giao hàng và chất lượng có thể cần nhiều “góc nhìn” attention. Hơn nữa, lớp hiện tại chỉ trả về context vector, không trả  $\alpha_t$ , nên project chưa thể vẽ heatmap từ quan trọng dù attention về lý thuyết hỗ trợ diễn giải. BiLSTM cũng xử lý tuần tự và cần thấy toàn câu, không phù hợp bằng mô hình nhân quả nếu yêu cầu streaming theo token.

# Chương 4

## Hai mô hình đối chứng

### 4.1 SimpleRNN baseline

SimpleRNN có luồng xử lý:

Embedding  $\rightarrow$  Dropout  $\rightarrow$  SimpleRNN(128)  $\rightarrow$  Dense(64)  $\rightarrow$  Dropout  $\rightarrow$  Softmax(5).

Tại mỗi bước thời gian  $t$ , trạng thái ẩn (hidden state) được cập nhật thông qua phương trình đệ quy cơ bản:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b). \quad (4.1)$$

Trong đó,  $W_x$  và  $W_h$  là các ma trận trọng số biểu diễn quá trình tích hợp thông tin từ từ vựng hiện tại  $x_t$  và ngữ cảnh quá khứ  $h_{t-1}$ .

Mô hình có 2.709.393 tham số, ít nhất trong ba kiến trúc. Việc bật cờ `unroll=True` giúp trải phẳng vòng lặp (unrolling) trên đồ thị tính toán theo chiều dài cố định 200, tối ưu hóa tốc độ nội suy tùy thuộc vào backend. Tuy nhiên, điểm yếu chí tử của SimpleRNN nằm ở chính phương trình đệ quy của nó: khi áp dụng thuật toán lan truyền ngược qua thời gian (BPTT - Backpropagation Through Time) trên một chuỗi dài 200 tokens, các đạo hàm phải nhân liên tiếp với ma trận  $W_h$ . Điều này dẫn đến sự bùng nổ (exploding) hoặc triệt tiêu đạo hàm (vanishing gradient) cực kỳ nghiêm trọng, khiến mạng gần như bất lực trong việc duy trì ngữ cảnh ở những từ đầu tiên trong câu.

Thực tế huấn luyện cho thấy đường validation của SimpleRNN dao động cực kỳ mạnh: độ chính xác (accuracy) có lúc tụt thẳng đứng xuống 49,72% rồi bật lên trên 72%. Hiện tượng bất định này là minh chứng rõ rệt nhất cho sự sụp đổ gradient của vanilla RNN khi phải vật lộn với nhiễu dữ liệu và Dropout 0,5. Trạng thái tốt nhất được lưu lại theo checkpoint có validation loss thấp nhất.

## 4.2 Transformer Encoder baseline

Transformer [4] là một bước ngoặt thay đổi hoàn toàn cách biểu diễn chuỗi bằng cách loại bỏ hoàn toàn cơ chế đệ quy, thay vào đó sử dụng Self-Attention để quan sát toàn cục. Luồng xử lý của khối Transformer như sau:

Token + Position Embedding  $\rightarrow$  1 Transformer block  $\rightarrow$  Masked Average Pooling  $\rightarrow$  Dense(64)  $\rightarrow$  Softmax(5).

Vì mô hình xử lý toàn bộ chuỗi song song nên khái niệm "thứ tự thời gian" bị xóa bỏ. Do đó, tín hiệu vị trí (Positional Encoding) bắt buộc phải được cộng trực tiếp vào Token Embedding. Lỗi sức mạnh của khối encoder nằm ở phương trình **Scaled Dot-Product Attention**:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (4.2)$$

Trong đó, Query ( $Q$ ), Key ( $K$ ) và Value ( $V$ ) đều được ánh xạ tuyến tính từ cùng một nguồn biểu diễn đầu vào. Phép nhân ma trận  $QK^T$  tạo ra một bản đồ tương quan giữa mọi cặp từ vựng trong câu bất chấp khoảng cách vật lý của chúng. Yếu tố tỷ lệ  $\frac{1}{\sqrt{d_k}}$  đóng vai trò vô cùng quan trọng giúp giữ bình ổn phương sai của tích vô hướng, tránh việc hàm softmax bị đẩy vào vùng bão hòa gradient.

Mô hình triển khai **Multi-Head Attention** với 4 head độc lập (mỗi head có số chiều chiếu  $\text{key\_dim} = 25$  để giữ nguyên tổng độ dài không gian là 100). Việc dùng nhiều head cho phép mạng linh hoạt nắm bắt đồng thời các mối quan hệ ngữ pháp và ngữ nghĩa khác nhau ở các không gian con (subspaces) riêng biệt. Lớp Feed-Forward Network (FFN) theo sau có cấu trúc mở rộng không gian  $100 \rightarrow 256 \rightarrow 100$ , tích hợp hàm kích hoạt GELU cùng cơ chế Residual Connection và Layer Normalization để chống suy biến mạng sâu. Tổng số lượng tham số là 2.790.645.

Về mặt lý thuyết, khả năng đổi chiều song song  $O(N^2)$  của Transformer giúp nó vượt trội hoàn toàn về tốc độ chạy và khả năng phát hiện liên kết xa so với LSTM. Tuy nhiên, sức mạnh này đi kèm một cái giá: Transformer khát dữ liệu (data-hungry) và rất dễ học lệch cấu trúc nếu thiếu đi một pha tiền huấn luyện (pre-training) khổng lồ như BERT. Kết quả thực nghiệm phản ánh chính xác điểm yếu này: sau epoch 3, tập huấn luyện bắt đầu "học vẹt" (loss giảm mạnh xuống 0,5077) trong khi validation loss lại bật ngược lên 1,0037. Mạng đã bị quá khớp (overfitting) nghiêm trọng và EarlyStopping phải buộc dừng sớm ở epoch thứ 6.

## 4.3 Điều kiện so sánh

Ba mô hình dùng chung:

- cùng tập train/validation, seed 42 và tokenizer fit trên train;
- cùng GloVe 100d, batch size 128 và độ dài chuỗi 200;

- cùng loss, Adam, Dropout 0,5, L2 0,01, EarlyStopping và scheduler;
- cùng trạng thái `use_class_weights=true`.

Sự thống nhất này làm phép so sánh kiến trúc có ý nghĩa hơn. Dù vậy, số epoch thực tế khác nhau do EarlyStopping: Transformer dừng ở 6 epoch, còn RNN và BiLSTM + Attention chạy đủ 10 epoch.

## 4.4 Tóm tắt khác biệt kiến trúc

Bảng 4.1: So sánh định tính ba kiến trúc

| Tiêu chí                            | SimpleRNN          | BiLSTM + Attention                    | Transformer Encoder              |
|-------------------------------------|--------------------|---------------------------------------|----------------------------------|
| <b>Vai trò</b>                      | Baseline đơn giản  | <b>Mô hình chính</b>                  | Baseline self-attention          |
| <b>Ngữ cảnh</b>                     | Một chiều          | Hai chiều                             | Hai chiều trong toàn chuỗi       |
| <b>Phụ thuộc xa</b>                 | Yếu                | Tốt hơn nhờ LSTM                      | Trực tiếp qua self-attention     |
| <b>Tổng hợp chuỗi</b>               | Hidden state cuối  | Additive attention                    | Masked average pooling           |
| <b>Song song hóa theo token</b>     | Thấp               | Thấp                                  | Cao                              |
| <b>Chi phí theo độ dài</b>          | Xấp xỉ tuyến tính  | Xấp xỉ tuyến tính                     | Self-attention bậc hai           |
| <b>Khả năng giải thích hiện tại</b> | Thấp               | Có tiềm năng, nhưng chưa xuất weights | Attention map phức tạp           |
| <b>Rủi ro chính</b>                 | Vanishing gradient | Chậm, bottleneck một context          | Overfit, cần dữ liệu/pretraining |

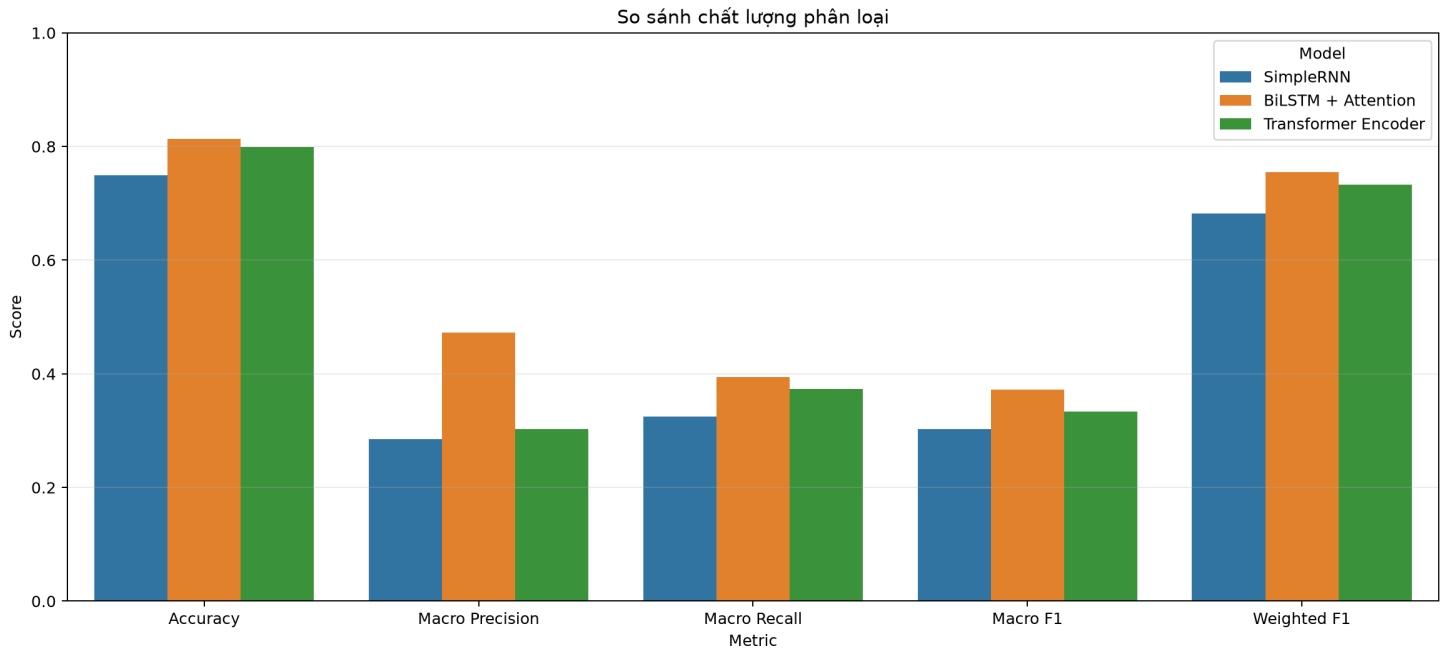
## Chương 5

# Kết quả và so sánh chi tiết ba mô hình

### 5.1 Kết quả tổng hợp

Bảng 5.1: Kết quả trên cùng tập validation 4.081 mẫu

| Mô hình                   | Accuracy      | Macro P       | Macro R       | Macro F1      | Weighted F1   |
|---------------------------|---------------|---------------|---------------|---------------|---------------|
| SimpleRNN                 | 0,7493        | 0,2847        | 0,3252        | 0,3029        | 0,6818        |
| <b>BiLSTM + Attention</b> | <b>0,8138</b> | <b>0,4722</b> | <b>0,3946</b> | <b>0,3727</b> | <b>0,7549</b> |
| Transformer Encoder       | 0,7988        | 0,3026        | 0,3735        | 0,3336        | 0,7328        |



Hình 5.1: So sánh các metric chất lượng tổng hợp

BiLSTM + Attention đứng đầu ở toàn bộ metric chất lượng:

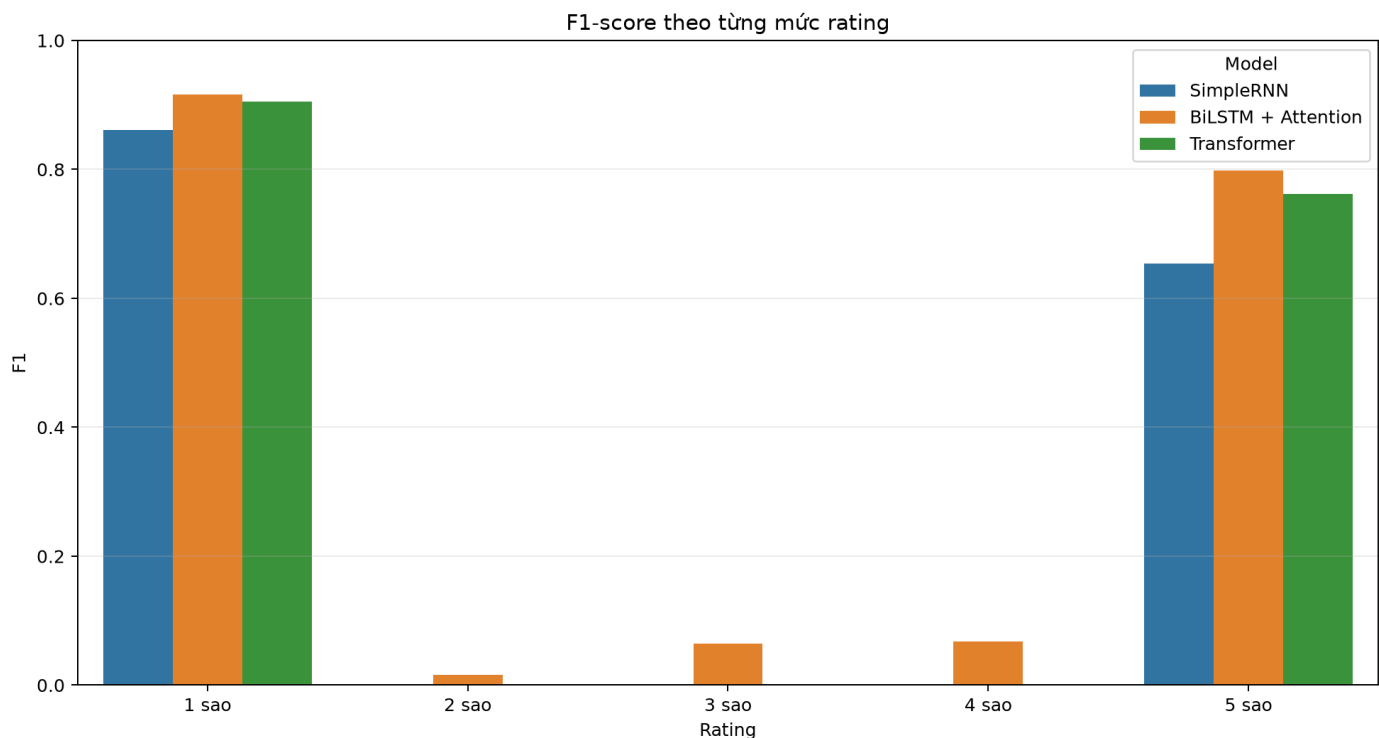
- hơn SimpleRNN 6,44 điểm phần trăm Accuracy;
- Macro F1 cao hơn SimpleRNN 23,05% theo tỷ lệ tương đối;
- hơn Transformer 1,49 điểm phần trăm Accuracy;
- Macro F1 cao hơn Transformer 11,74%;

Kết quả này ủng hộ lựa chọn mô hình chính. BiLSTM giúp giữ ngữ cảnh tốt hơn SimpleRNN; attention tổng hợp chuỗi linh hoạt hơn việc chỉ dùng hidden state cuối; đồng thời mô hình ổn định hơn Transformer nhờ huấn luyện không pretrain.

Tuy nhiên, Macro F1 tốt nhất vẫn chỉ là 0,3727, thấp hơn nhiều so với Accuracy 0,8138. Chênh lệch này là bằng chứng định lượng rằng hiệu năng đang tập trung ở các lớp lớn.



## 5.2 So sánh F1 theo từng mức rating



Hình 5.2: F1-score theo từng lớp rating

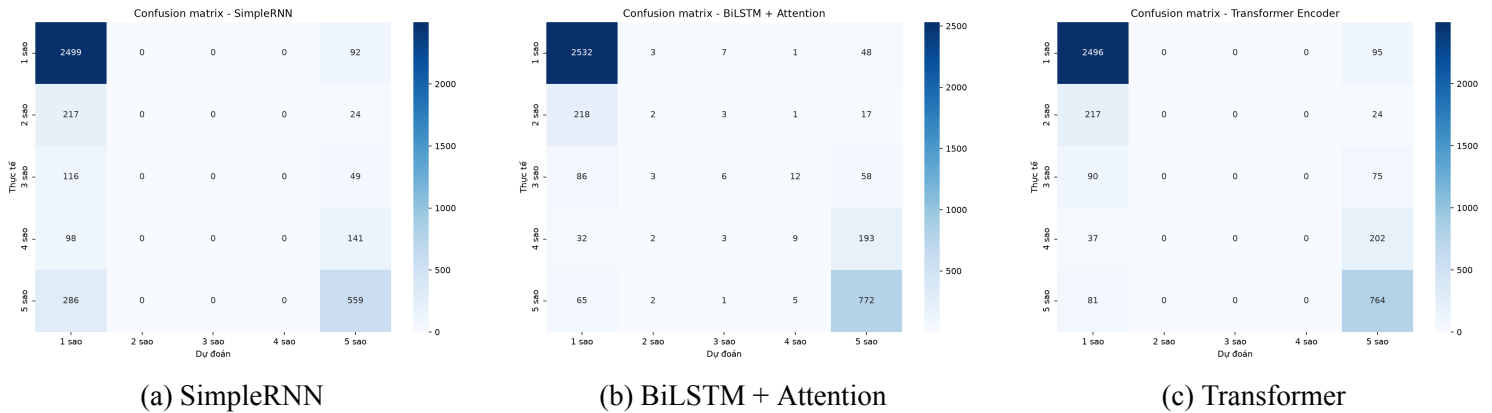
Bảng 5.2: F1-score từng lớp

| Mô hình            | 1 sao         | 2 sao         | 3 sao         | 4 sao         | 5 sao         |
|--------------------|---------------|---------------|---------------|---------------|---------------|
| SimpleRNN          | 0,8607        | 0,0000        | 0,0000        | 0,0000        | 0,6538        |
| BiLSTM + Attention | <b>0,9167</b> | <b>0,0158</b> | <b>0,0649</b> | <b>0,0674</b> | <b>0,7988</b> |
| Transformer        | 0,9057        | 0,0000        | 0,0000        | 0,0000        | 0,7621        |

Biểu đồ cho thấy cả ba mô hình thực tế chủ yếu giải một bài toán gần giống phân loại hai cực. SimpleRNN và Transformer có F1 bằng 0 ở cả ba lớp 2, 3 và 4 sao. BiLSTM + Attention là mô hình duy nhất tạo ra một số true positive ở các lớp này, nhưng mức cải thiện còn nhỏ về giá trị tuyệt đối.

F1 lớp 3 và 4 của mô hình chính cao hơn lớp 2, nhưng đều dưới 0,07. Vì vậy, không nên mô tả rằng mô hình đã “bắt tốt class thiểu số”. Mô tả đúng hơn là: *BiLSTM + Attention giảm mức độ sụp đổ lớp so với hai baseline, nhưng chưa khắc phục được mất cân bằng.*

## 5.3 So sánh confusion matrix



Hình 5.3: Confusion matrix của ba mô hình

SimpleRNN và Transformer chỉ sử dụng hai cột dự đoán 1 sao và 5 sao. Điều này không phải lỗi hiển thị: các cột 2–4 thực sự bằng 0. Transformer tốt hơn RNN chủ yếu do phân tách hai cực chính xác hơn, không phải vì hiểu tốt hơn các mức trung gian.

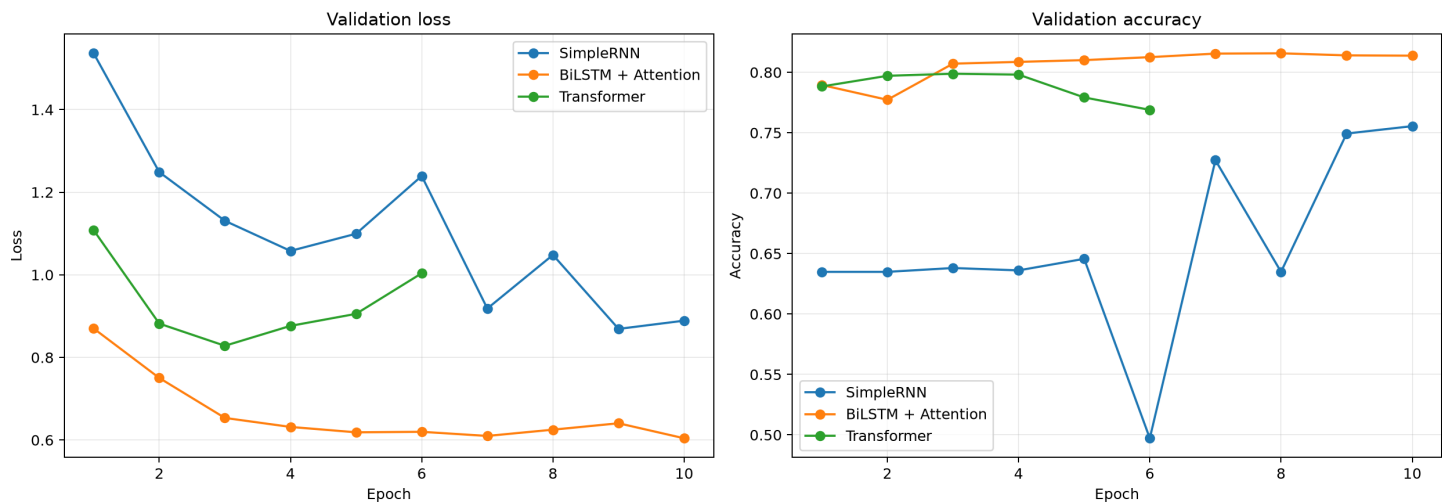
BiLSTM + Attention sử dụng đủ năm cột, nhưng phân bố dự đoán vẫn lệch:

Bảng 5.3: Phân bố nhãn dự đoán của BiLSTM + Attention

| Nhãn dự đoán | Số mẫu | Tỷ lệ  |
|--------------|--------|--------|
| 1 sao        | 2.933  | 71,87% |
| 2 sao        | 12     | 0,29%  |
| 3 sao        | 20     | 0,49%  |
| 4 sao        | 28     | 0,69%  |
| 5 sao        | 1.088  | 26,66% |

Các lỗi chủ yếu có cấu trúc hợp lý theo thứ tự rating: 2 sao bị kéo về 1 sao, 4 sao bị kéo về 5 sao, còn 3 sao chia sang cả hai cực. Điều này cho thấy mô hình học được polarity nhưng decision boundary cho cường độ sentiment chưa đủ tinh.

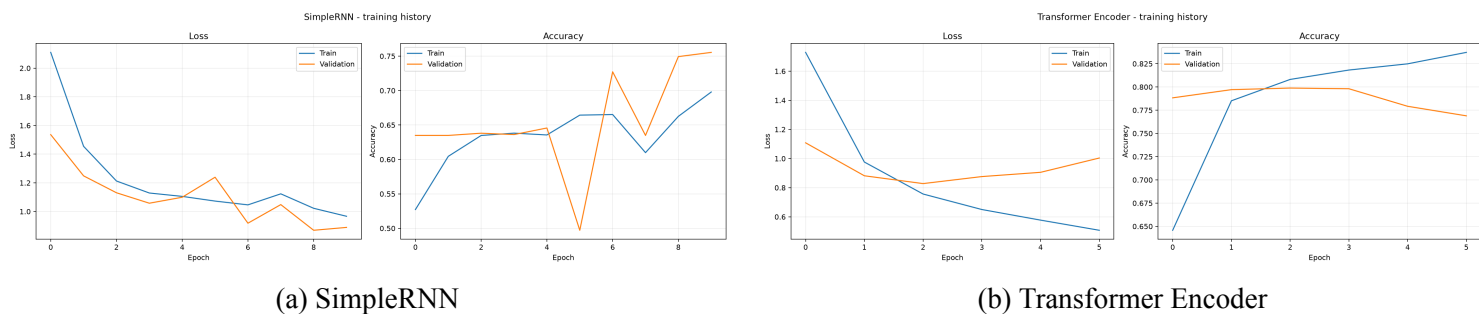
## 5.4 Đường validation của ba mô hình



Hình 5.4: Validation loss và validation accuracy của ba mô hình

BiLSTM + Attention có đường validation ổn định nhất: loss giảm nhanh và duy trì quanh 0,60–0,64. Transformer hội tụ nhanh trong ba epoch đầu nhưng validation loss tăng sau đó, trong khi SimpleRNN dao động mạnh.

Cần lưu ý rằng mỗi đường chỉ có một lần chạy với seed 42. Không có dải độ lệch chuẩn hoặc confidence interval. Một đường đẹp hơn trong một split chưa đủ để kết luận độ ổn định thống kê. Thực nghiệm đầy đủ nên chạy nhiều seed và báo cáo trung bình  $\pm$  độ lệch chuẩn.

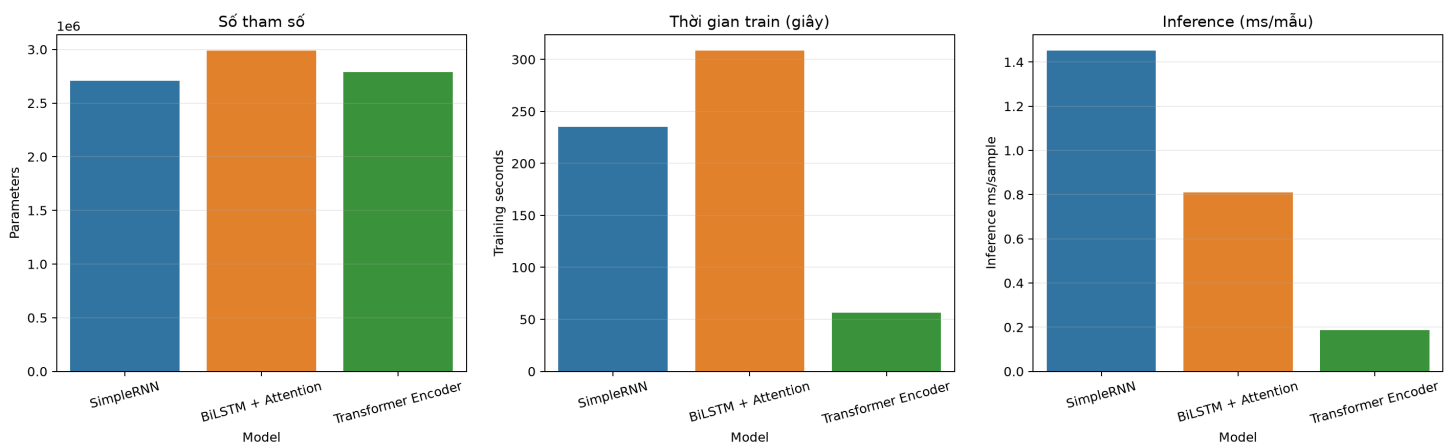


Hình 5.5: Training history chi tiết của hai mô hình đối chứng

## 5.5 Chi phí tài nguyên

Bảng 5.4: Số tham số và thời gian ghi nhận trong artifact

| Mô hình             | Tham số          | Epoch | Train (giây) | Inference (ms/mẫu) |
|---------------------|------------------|-------|--------------|--------------------|
| SimpleRNN           | <b>2.709.393</b> | 10    | 235,36       | 1,4516             |
| BiLSTM + Attention  | 2.988.817        | 10    | 308,47       | 0,8103             |
| Transformer Encoder | 2.790.645        | 6     | <b>56,42</b> | <b>0,1859</b>      |



Hình 5.6: So sánh số tham số, thời gian train và thời gian inference

Transformer nhanh nhất: thời gian train được ghi nhận nhanh hơn BiLSTM khoảng 5,47 lần và inference nhanh hơn khoảng 4,36 lần. Điều này phù hợp với khả năng song song hóa self-attention. BiLSTM chậm nhất khi train do phải chạy LSTM theo hai hướng.

Tuy nhiên, không nên xem các con số thời gian là hằng số của kiến trúc. Chúng phụ thuộc GPU, phiên bản TensorFlow, batch size, warm-up, I/O và số epoch. SimpleRNN có inference chậm hơn BiLSTM trong lần chạy này dù kiến trúc đơn giản hơn; đây có thể là hệ quả của implementation `unroll=True`, kernel backend và overhead đo. Kết luận an toàn là Transformer nhanh nhất *trong artifact hiện tại*, không phải SimpleRNN luôn chậm hơn BiLSTM.

## 5.6 So sánh trực tiếp BiLSTM + Attention với SimpleRNN

Hai mô hình cùng dùng recurrence, nhưng khác ở ba điểm:

1. LSTM có cổng và cell state, SimpleRNN chỉ có một hidden state.

2. BiLSTM đọc cả hai chiều, SimpleRNN chỉ đọc từ trái sang phải.
3. Attention của mô hình chính tổng hợp toàn bộ chuỗi, SimpleRNN dùng output cuối.

Chênh lệch 6,44 điểm Accuracy cho thấy ba thay đổi này có giá trị thực nghiệm rõ ràng. Đặc biệt, chỉ BiLSTM + Attention tạo được true positive cho lớp 2–4 sao. Đổi lại, mô hình chính có thêm khoảng 279 nghìn tham số và mất thêm 73 giây huấn luyện trong lần chạy.

## 5.7 So sánh trực tiếp BiLSTM + Attention với Transformer

Transformer gần mô hình chính hơn về Accuracy nhưng yếu hơn về Macro Precision, Macro F1. Transformer nhận diện hai cực tốt nhưng co cụm hoàn toàn, trong khi BiLSTM còn duy trì một lượng nhỏ dự đoán ở lớp giữa. Với dataset hiện tại, inductive bias tuần tự và cổng nhớ của LSTM có vẻ phù hợp hơn một Transformer encoder nhỏ huấn luyện không pretrain.

Ngược lại, Transformer vượt trội về tốc độ. Nếu mục tiêu triển khai ưu tiên throughput và bài toán có thể rút về nhị phân, Transformer nhỏ là lựa chọn đáng cân nhắc. Nếu mục tiêu giữ đủ năm lớp và tối ưu chất lượng hiện có, BiLSTM + Attention là lựa chọn hợp lý hơn, nhưng bắt buộc phải xử lý mất cân bằng.

## 5.8 Mô hình được lựa chọn

Dựa trên kết quả hiện tại, project lựa chọn **BiLSTM + Attention** làm mô hình chính vì:

- đứng đầu Accuracy, Macro Precision, Macro Recall, Macro F1 và Weighted F1;
- là mô hình duy nhất không sụp hoàn toàn về hai nhãn;
- learning curve ổn định hơn;
- kiến trúc phù hợp mục tiêu nghiên cứu về ngữ cảnh hai chiều và attention.

Lựa chọn này là tương đối trong ba mô hình, không che giấu rằng Macro F1 0,3727 vẫn thấp và lớp giữa chưa đạt chất lượng sử dụng thực tế.

# Chương 6

## Vấn đề tồn tại, hướng cải thiện và kết luận

### 6.1 Vấn đề nghiêm trọng nhất: mất cân bằng và sụp đổ lớp

Dữ liệu có 63,49% lớp 1 sao nhưng class weights bị tắt. Cross-entropy tối ưu trung bình trên mẫu, nên gradient từ lớp 1 sao áp đảo lớp 2–4. Một chiến lược dự đoán chủ yếu 1 và 5 sao vẫn đạt Accuracy cao. Đây là nguyên nhân trực tiếp nhất giải thích sự chênh lệch giữa Accuracy và Macro F1.

Mã nguồn đã có khả năng tính class weight bằng `compute_class_weight(class_weight="balanced")`; bước đầu tiên cần làm là bảo đảm `USE_CLASS_WEIGHTS=1` trong lần train chính thức và lưu lại weights vào artifact. Sau đó phải so sánh:

- không class weight;
- class weight cân bằng;
- oversampling lớp 2–4;
- focal loss hoặc class-balanced focal loss.

Metric chọn checkpoint cũng nên cân nhắc Macro F1 thay vì chỉ `val_loss`. Nếu mục tiêu là năm lớp cân bằng, validation loss trung bình có thể tiếp tục ưu tiên lớp lớn.

### 6.2 Bài toán có thứ bậc nhưng loss đang xem nhỡ là danh mục

Rating 1–5 có quan hệ thứ tự, nhưng softmax cross-entropy coi năm lớp là năm category độc lập. Trong loss, dự đoán 1 sao thành 2 sao và 1 sao thành 5 sao đều chỉ là “sai lớp”. Rating MAE chỉ được dùng sau huấn luyện, chưa tác động vào gradient.

Các hướng phù hợp hơn gồm:

1. ordinal regression với các ngưỡng tích lũy  $P(y > 1), \dots, P(y > 4)$ ;
2. Earth Mover's Distance hoặc loss dựa trên khoảng cách rating;
3. label smoothing có cấu trúc, phân bố một phần xác suất cho lớp lân cận.

Những loss này khuyến khích mô hình hiểu 3 sao nằm giữa 1 và 5 sao, thay vì chỉ học năm nhãn rời rạc.

## 6.3 Hạn chế của quy trình đánh giá

1. **Không có test set độc lập.** Validation được dùng cho EarlyStopping và báo cáo, nên kết quả có thể lạc quan.
2. **Chỉ một split và một seed.** Chưa biết mức biến thiên của kết quả.
3. **Không có confidence interval.** Chênh lệch 1,49 điểm Accuracy giữa BiLSTM và Transformer chưa được kiểm định thống kê.
4. **Thời gian chưa chuẩn hóa đầy đủ.** Chưa lưu loại GPU, phiên bản TensorFlow, số lần warm-up và peak memory.
5. **Không có error analysis theo độ dài hoặc từ OOV.** Chưa biết mô hình thất bại nhiều hơn ở review dài, ngắn hay chứa từ miền chuyên biệt.

Thiết kế thực nghiệm nên chuyển sang train/validation/test, ví dụ 70/15/15 hoặc 80/10/10, giữ test hoàn toàn đóng cho tới khi chốt mô hình. Nên chạy ít nhất 3–5 seed.

## 6.4 Hạn chế tiền xử lý

Regex hiện chỉ giữ chữ cái tiếng Anh a–z, chữ số và dấu nháy đơn. Điều này phù hợp phần lớn review tiếng Anh nhưng có thể làm mất emoji, ký hiệu nhấn mạnh, từ Unicode và thông tin dấu câu. Trong sentiment analysis, “!!!”, emoji hoặc viết hoa toàn bộ có thể là tín hiệu hữu ích.

Post-truncation tại 200 token bỏ toàn bộ phần cuối review dài. Nếu kết luận hoặc rating rationale thường xuất hiện ở cuối, đây là mất mát đáng kể. Cần thống kê phân bố độ dài và tỷ lệ review bị cắt trước khi chọn 200.

GloVe không được huấn luyện riêng cho miền Amazon và coverage không được lưu trong `metrics.json`. Các từ sản phẩm, model code và lỗi chính tả có thể bắt đầu bằng vector 0. Nên lưu số lượng hit/miss, thử FastText subword hoặc embedding pretrained theo miền.

## 6.5 Hạn chế riêng của BiLSTM + Attention

- Attention chỉ có một context vector, có thể trộn nhiều khía cạnh trái chiều trong review.
- Attention weights chưa được xuất ra, nên chưa có kiểm chứng định tính mô hình đang tập trung vào từ nào.
- $L2 = 0,01$  và  $\text{Dropout} = 0,5$  là regularization khá mạnh; cần ablation để xác định có làm underfit lớp thiếu số hay không.
- BiLSTM tuần tự nên chậm hơn Transformer khi chuỗi dài.
- Không có recurrent dropout trong LSTM và không có normalization sau BiLSTM; đây là các lựa chọn có thể thử nhưng cần đánh giá thực nghiệm.

Để phân tích attention, layer có thể trả đồng thời  $(c, \alpha)$ . Sau đó vẽ màu trên token cho các ca đúng, ca sai và review chứa mệnh đề đối lập. Cần thận trọng: attention weight là tín hiệu diễn giải hữu ích, nhưng không tự động đồng nghĩa với giải thích nhân quả.

## 6.6 Hạn chế riêng của hai baseline

**SimpleRNN** gặp dao động huấn luyện, không giữ phụ thuộc dài và bỏ hoàn toàn ba lớp giữa. Cờ `unroll=True` với chuỗi 200 có thể tăng memory và không đảm bảo nhanh hơn trên mọi phần cứng. Baseline vẫn hữu ích để chứng minh lợi ích của LSTM, nhưng không phù hợp để triển khai bài toán hiện tại.

**Transformer** overfit sau khoảng ba epoch và cũng bỏ hoàn toàn lớp 2–4. Một encoder block 100 chiều không pretrained khó học đồng thời ngữ pháp, ngữ nghĩa miền và decision boundary năm lớp từ dataset nhỏ. Có thể thử pretraining như DistilBERT, hoặc tăng dữ liệu và dùng warm-up/weight decay; tuy nhiên khi đó chi phí và phạm vi so sánh sẽ thay đổi đáng kể.



## 6.7 Lộ trình cải thiện đề xuất

Bảng 6.1: Thứ tự ưu tiên cải thiện project

| Ưu tiên | Thay đổi                                        | Mục tiêu                                                      |
|---------|-------------------------------------------------|---------------------------------------------------------------|
| 1       | <b>Bật class weights và retrain đủ ba model</b> | Kiểm tra trực tiếp nguyên nhân sụp lớp 2–4.                   |
| 2       | <b>Dùng Macro F1 làm metric chọn model</b>      | Không để lớp 1 sao chi phối checkpoint.                       |
| 3       | <b>Tạo test set độc lập, chạy nhiều seed</b>    | Có ước lượng tổng quát hóa và độ bất định đáng tin cậy.       |
| 4       | <b>Thử ordinal loss</b>                         | Khai thác cấu trúc thứ tự 1–5 sao và giảm lỗi nhảy nhiều mức. |
| 5       | <b>Xuất attention weights</b>                   | Phân tích token quan trọng và phát hiện shortcut.             |
| 6       | <b>Tuning dropout, L2, units, max length</b>    | Giảm underfit/overfit bằng ablation có kiểm soát.             |
| 7       | <b>Thử pretrained Transformer hoặc FastText</b> | Cải thiện biểu diễn từ và từ ngoài vocabulary.                |

## 6.8 Kết luận

Trong thiết lập hiện tại, BiLSTM + Attention là mô hình tốt nhất trong ba kiến trúc. Mô hình đạt Accuracy 81,38% và Macro F1 0,3727; vừa chính xác hơn, vừa tạo lỗi rating nhỏ hơn SimpleRNN và Transformer. Kiến trúc 128 đơn vị mỗi chiều kết hợp additive attention có mask là lựa chọn hợp lý cho review độ dài vừa, và learning curve cho thấy quá trình huấn luyện tương đối ổn định.

Kết luận quan trọng hơn là project chưa giải quyết tốt phân loại năm mức. Accuracy cao chủ yếu đến từ hai lớp 1 và 5 sao. Class weights bị tắt trong dữ liệu lệch mạnh làm cả ba mô hình co cụm về hai cực; BiLSTM + Attention chỉ giảm nhẹ, chưa loại bỏ hiện tượng này. Do đó, bước tiếp theo không nên chỉ tăng kích thước mô hình. Ưu tiên đúng là sửa thiết kế thực nghiệm: cân bằng lớp, dùng metric/loss phù hợp với lớp thiểu số và thứ tự rating, tách test set, chạy nhiều seed và kiểm tra attention. Khi các bước đó được thực hiện, giá trị thật sự của kiến trúc BiLSTM + Attention mới có thể được đánh giá công bằng.

# **TÀI LIỆU THAM KHẢO**

# Tài liệu tham khảo

- [1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] M. Schuster and K. K. Paliwal, “Bidirectional Recurrent Neural Networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” *International Conference on Learning Representations*, 2015.
- [4] A. Vaswani et al., “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] J. Pennington, R. Socher, and C. D. Manning, “GloVe: Global Vectors for Word Representation,” *Proceedings of EMNLP*, pp. 1532–1543, 2014.
- [6] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *International Conference on Learning Representations*, 2015.
- [7] Amazon Reviews Dataset, Kaggle, <https://www.kaggle.com/dongrelaxman/amazon-reviews-dataset>, truy cập trong quá trình thực hiện project.
- [8] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” 2015. <https://www.tensorflow.org/>.